

심화 실습 평가 2

TCP 를 이용한 원격 장치 제어 프로그램

VEDA 한화비전 교육과정 4 기

신채훈

1. 프로젝트 개요

1.1 프로젝트 목적

본 프로젝트는 TCP 소켓 통신을 기반으로 한 원격 장치 제어 시스템을 구현하는 것을 목표로 합니다. 라즈베리파이 4 를 서버로, Ubuntu Linux 를 클라이언트로 사용하여 LED, 부저, 조도센서, 7 세그먼트 디스플레이 등 다양한 하드웨어 장치를 원격으로 제어하는 프로그램입니다.

1.2 시스템 구성

구분	사양	역할
서버	Raspberry Pi 4	하드웨어 장치 제어, TCP 서버, 데몬 프로세스
클라이언트	Ubuntu Linux	사용자 인터페이스, TCP 클라이언트, 장치 제어 명령 전송
통신 프로토콜	TCP/IP	신뢰성 있는 원격 제어 명령 전송

1.3 제어 대상 장치

장치	기능 설명
LED	on/off 제어 및 밝기(최대/중간/최저) 조절 제어 (PWM, BCM12 핀)
부저 (Buzzer)	음악 소리 on/off 제어
조도센서 (CDS)	조도 값 실시간 확인, 빛 감지 여부에 따른 LED 자동 제어
7 세그먼트 (FND)	0~9 숫자 표시, 1 초 단위 카운트다운, 0 도달 시 부저 울림 (74LS47 디코더 사용)

2. 개발 일정

일정	작업 항목	세부 내용	상태
6/2	프로젝트 설계	TCP 통신 설계, 공유 라이브러리 구조 설계, 하드웨어 핀 맵 확인	완료
6/2	서버 구현	데몬 프로세스 구성, TCP 핸들러, 동적 라이브러리 로드(dlopen)	완료
6/2-6/4	장치 라이브러리	LED, 부저, 조도센서, 7 세그먼트 .so 파일 구현 (wiringPi)	완료
6/4	클라이언트 구현	메뉴 UI, TCP 소켓 통신, 시그널 처리 (INT 신호)	완료
6/5	빌드 자동화	Makefile 작성 (전체 빌드, .so 빌드, clean 타겟)	완료
6/5	테스트 및 문서화	실행 과정 캡처, README 작성, 개발 문서 작성	완료

3. 세부 구현 내용

3.1 전체 아키텍처

본 프로젝트는 클라이언트-서버 구조로 설계되었으며, 서버 측 장치 제어 로직은 동적 공유 라이브러리(.so) 형태로 분리 구현하여 장치별 독립적인 업그레이드가 가능하도록 구성하였습니다.

디렉토리 구조

```
프로젝트 루트/
├── code/
│   ├── client/
│   │   └── client_main.c      # 클라이언트 소스
│   ├── server/
│   │   ├── server_main.c     # 서버 메인 (데몬)
│   │   ├── tcp_handler.c     # TCP 처리 모듈
│   │   └── lib_src/
│   │       ├── led.c         # LED 제어 라이브러리
│   │       ├── buzzer.c      # 부저 제어 라이브러리
│   │       ├── cds.c         # 조도센서 라이브러리
│   │       └── fnd.c         # 7 세그먼트 라이브러리
│   └── exec/
│       ├── client/client_app # 클라이언트 실행 파일
│       ├── server/server_main # 서버 실행 파일
│       └── lib/
│           └── libled.so
```

```
|
|   └─ libbuzzer.so
|   └─ libcnd.so
|   └─ libfnd.so
└─ docs/
    └─ manual.pdf
    └─ README.md
└─ misc/                                # 회로도 등 기타 파일
└─ Makefile
└─ .gitignore
```

3.2 서버 구현 (Raspberry Pi 4)

3.2.1 데몬 프로세스

서버는 Daemon 프로세스 형태로 구동되어 백그라운드에서 지속적으로 TCP 연결을 대기합니다.

- `daemon(1, 0)` 을 통해 프로세스 데몬화
- `fork()`를 통해 자식 프로세스 생성 후 부모 프로세스 종료
- `setsid()`를 통해 새로운 세션 리더가 되어 터미널로부터 분리
- 표준 입출력(stdin/stdout/stderr)을 /dev/null 로 리다이렉션

3.2.2 TCP 서버 소켓

tcp_handler.c 에서 TCP 소켓 생성, 바인딩, 연결 수락 등 네트워크 처리를 담당합니다.

- 서버 포트: TCP 소켓 바인딩 및 `listen()` 대기
- 클라이언트 연결 시 멀티스레드(pthread) 또는 멀티프로세스 방식으로 장치 제어 처리
- `-lpthread`, `-ldl` 링크 옵션 사용

3.2.3 동적 라이브러리 로딩 (dlopen)

장치 제어 기능은 공유 라이브러리(.so) 형태로 분리 구현하여, 필요 시 해당 .so 파일만 교체/업로드하여 기능 업그레이드가 가능합니다.

```
// 동적 라이브러리 로드 예시
void *handle = dlopen("exec/lib/libled.so", RTLD_LAZY);
void (*led_control)(int, int) = dlsym(handle, "led_control");
led_control(LED_ON, BRIGHTNESS_MAX);
dlclose(handle);
```

3.3 장치 제어 라이브러리 (.so)

3.3.1 LED 제어 (libled.so)

- BCM12 핀 (PWM0_0) 사용
- on/off 제어 및 PWM 을 통한 밝기 3 단계 제어 (최대/중간/최저)
- wiringPi 라이브러리 사용 (`-lwiringPi`)
- 컴파일 옵션: `-fPIC -shared`

3.3.2 부저 제어 (libbuzzer.so)

- 클라이언트로부터 수신된 명령에 따라 음악(멜로디) 재생 on/off
- 7 세그먼트 카운트다운 종료(0) 시 부저 울림 연동
- wiringPi 라이브러리 사용

3.3.3 조도센서 제어 (libcnd.so)

- 조도 센서(CDS) 값을 클라이언트로 전송
- 빛 감지 여부에 따른 LED 자동 제어: 빛 없음 → LED ON, 빛 감지 → LED OFF
- wiringPi 라이브러리 사용

3.3.4 7 세그먼트 제어 (libfnd.so)

- 74LS47 BCD-to-7-Segment 디코더 IC 사용
- 클라이언트로부터 전송된 숫자(0~9)를 초 단위 시작값으로 표시
- 1 초마다 1 씩 감소 카운트다운 (스레드/타이머 기반)
- 0 도달 시 부저 울림 트리거

3.3.5 추가 기능 : 클라이언트에서 요청하여 계수기(counter) 증가 기능 (libfnd.so)

- 클라이언트에서 명령어 10 을 받으면 7 세그먼트 값을 1 씩 증가
- 9 에서 명령어 10 을 받으면 0 으로 초기화

3.4 클라이언트 구현 (Ubuntu Linux)

3.4.1 사용자 인터페이스

클라이언트는 텍스트 기반 메뉴 UI 로 구성되며, 사용자 입력에 따라 해당 장치 제어 명령을 서버로 전송합니다.

```
$ ./exec/client/client_app

[ Device Control Menu ]
1. LED ON
2. LED OFF
3. Set Brightness
4. BUZZER ON (play melody)
5. BUZZER OFF (stop)
6. SENSOR ON (감지 시작)
7. SENSOR OFF (감지 종료)
8. SEGMENT DISPLAY (숫자 표시 후 카운트다운)
9. SEGMENT STOP (카운트다운 중단)
+ 10. SEGMENT INC (수동 카운트 증가)
0. Exit
Select:
```

3.4.2 시그널 처리

클라이언트 프로그램 실행 중 강제 종료되지 않도록 시그널 핸들러를 등록하였습니다.

- SIGINT, SIGTERM, SIGPIPE 신호 처리: 안전하게 소켓 종료 후 프로그램 종료

```
signal(SIGINT, handler_sigint); // INT 신호

Signal(SIGTERM, handle_sigint); // kill 명령에 죽도록

Signal(SIGPIPE, SIG_IGN); // 네트워크 단절에 안전하게 종료되도록
```

3.5 빌드 자동화 (Makefile)

프로젝트 전체 빌드는 루트 디렉토리의 Makefile 로 자동화되어 있습니다.

명령어	설명
make 또는 make all	클라이언트, 서버 실행 파일 및 모든 .so 공유 라이브러리 빌드
make libs	장치 제어 .so 파일만 빌드 (libled.so, libbuzzer.so, libcnd.so, libfnd.so)
make clean	exec/ 하위 빌드 결과물 전체 삭제

주요 컴파일 옵션

- gcc -Wall -Wextra -O2: 경고 최대화 및 최적화 빌드
- -fPIC -shared: 위치 독립 코드(공유 라이브러리 필수)
- -lwiringPi: 라즈베리파이 GPIO 제어 라이브러리 링크
- -ldl -lpthread: 동적 라이브러리 로딩 및 멀티스레드 지원

4. 평가 기준 구현 체크리스트

4.1 장치 제어 구현 여부 (40 점)

장치	구현 내용	배점	구현 여부
LED	클라이언트에서 ON/OFF 및 밝기(최대/중간/최저) 조절 제어	10 점	[V]
부저	클라이언트에서 (음악) 소리 ON/OFF 제어	10 점	[V]
조도센서	클라이언트에서 조도 값 확인, 빛 감지 여부에 따른 LED 자동 제어	10 점	[V]
7 세그먼트	클라이언트에서 전송한 숫자(0~9) 표시, 1 초 카운트다운, 0 도달 시 부저 울림	10 점	[V]

4.2 구현 내용 (30 점)

항목	구현 여부	구현 방법
멀티 프로세스 또는 스레드 이용한 장치	[V]	pthread 사용 (server_main.c, fnd.c 카운트다운)

제어		스레드)
장치 제어 프로그램을 공유 라이브러리 형식으로 작성	[V]	.so 파일로 각 장치 라이브러리 분리 (libled.so 등)
동적 라이브러리 기능 이용 (dlopen)	[V]	서버에서 dlopen/dlsym/dlclose 로 .so 동적 로드
클라이언트 강제 종료 방지 시그널 처리	[V]	SIGINT 핸들러 등록, 소켓 정리 후 안전 종료
서버 Daemon 프로세스 구성	[V]	fork + setsid 로 데몬화, 백그라운드 실행
빌드 자동화 (make 또는 cmake)	[V]	Makefile 로 전체 빌드, .so 개별 빌드, clean 구현

5. 문제점 및 보완 사항

5.1 개발 중 발생한 문제점

문제 1: 동적 라이브러리 경로 문제

dlopen() 호출 시 .so 파일의 절대 경로 또는 LD_LIBRARY_PATH 설정이 필요하였습니다.

→ 해결: exec/lib/ 경로를 기준으로 상대 경로 지정 및 실행 디렉토리 고정

문제 2: 카운트다운 스레드 동기화

7 세그먼트 카운트다운 중 SEGMENT STOP 명령 수신 시 스레드 안전 종료 처리가 필요하였습니다.

→ 해결: volatile 플래그 변수 사용 및 스레드 join() 처리

5.2 향후 보완 사항

- 클라이언트 UI 개선: ncurses 기반 TUI 적용으로 실시간 상태 표시
- 다중 클라이언트 지원: 스레드 풀 도입으로 동시 접속 처리 개선
- 보안 강화: 클라이언트 인증 메커니즘 추가 (토큰 기반 인증)
- 로깅 시스템: syslog 연동으로 서버 측 이벤트 로그 관리
- 네트워크 재연결: 클라이언트 측 자동 재연결 로직 추가

6. 빌드 및 실행 방법

6.1 사전 요구사항

- 서버(Raspberry Pi 4): wiringPi 라이브러리 설치 필요

```
sudo apt-get install wiringpi
```

- 클라이언트(Ubuntu): gcc, make 설치

```
sudo apt-get install build-essential
```

6.2 빌드

```
# 프로젝트 루트에서 실행
make          # 전체 빌드 (클라이언트 + 서버 + .so 라이브러리)
make libs     # .so 라이브러리만 빌드
make clean    # 빌드 결과물 삭제
```

6.3 실행

서버 실행 (Raspberry Pi)

```
./exec/server/server_main
```

클라이언트 실행 (Ubuntu)

```
./exec/client/client_app
```

6.4 GitHub 저장소

https://github.com/Hun0305/VEDA_AdvancedPractice2